

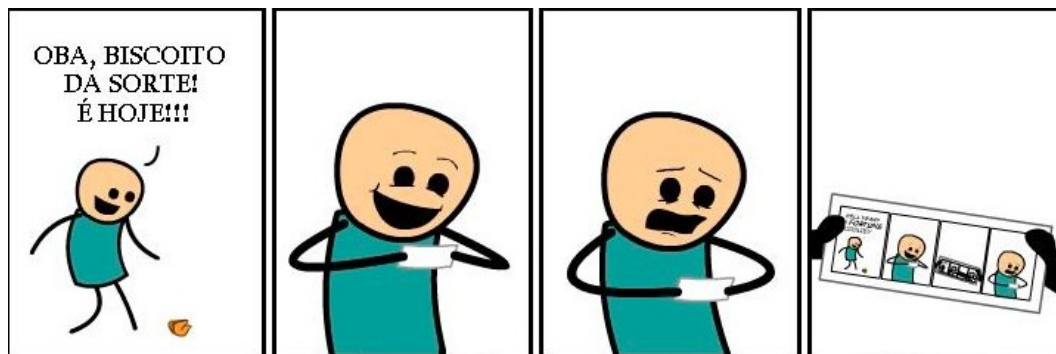


Instituto Politécnico
de Viana do Castelo

Programação I

Recursividade

Recursividade (ou recursão)





Existem dois métodos essenciais para a conceção e desenvolvimento de algoritmos: método iterativo e o método recursivo.

- **Método iterativo** – Este método de desenvolvimento de algoritmos tem por base a divisão do problema principal em sub-problemas. Determinados sub-problemas e respetivas tarefas têm um padrão de comportamento repetitivo o que leva à utilização de estruturas cíclicas de controlo do problema.
- **Método recursivo** – A recursividade é uma característica fundamental para a conceção e construção de algoritmos. Esta característica é interpretada como a capacidade de uma função (ou método) se invocar a si própria.



Para se aplicar a recursividade a um determinado problema, é necessário considerar:

- O caso base recursivo (de paragem) – existência de um conjunto de casos que não necessitam de operações significativas de computação.
- O passo recursivo – representação dos diferentes subprogramas que recursivamente chegarão ao caso base.



A Recursividade pode ser descrita como uma abordagem algorítmica em que uma função se invoca a ela própria. A recursividade pode ser de dois tipos:

- Direta - Quando uma função se invoca a ela mesma no seu corpo da função.
- Indireta - Quando uma função f invoca uma outra função g que por sua vez volta a invocar a função f .



Exemplo 1: Factorial

O factorial de um número n pode ser definido como

$$n! = 1 \times 2 \times 3 \times \dots \times n$$

ou

$$n! = n \times \dots \times 3 \times 2 \times 1$$

No modo iterativo o código seria algo como:

```
factorial = 1;
for (i = 1; i <= n; ++i) {
    factorial *= i;    // ou fatorial = fatorial * i
                      // Multiplicação de cada valor pelo valor anterior
}
```



Exemplo: A função factorial também poderia ser implementada de uma forma recursiva, como se pode ver no exemplo seguinte:

```
#include <stdio.h>

long factorial( int n ) {
    if (n<=0) return 1;
    return n* factorial(n-1);
}

int main() {
    int n=0;
    long result=0;
    printf("introduza um numero para calcular o fatorial\n");
    scanf("%d",&n);
    result=factorial(n);
    printf("o fatorial de %d é %ld\n",n,result);
    return 0;
}
```



Exemplo 2: O próximo programa calcula o somatório dos números inteiros positivos até ao número n por meio do método iterativo:

```
#include <stdio.h>

int soma( int n ) {
    int i,tot=0;
    for( i=1; i<=n; i++ ) {
        tot = tot + i;
    }
    return tot;
}

int main(){
    int n, result=0;
    printf("introduza um numero inteiro\n");
    scanf("%d",&n);
    result=soma(n);
    printf("A soma dos n° até %d é %d\n",n,result);
    return 0;
}
```



Exemplo 2: A função soma() também pode ser implementada usando recursividade, da seguinte forma:

```
#include <stdio.h>

int soma( int n ) {
    if (n==0) return 0;
    return n + soma(n-1);
}

int main(){
    int n, result=0;
    printf("introduza um numero inteiro\n");
    scanf("%d",&n);
    result=soma(n);
    printf("A soma dos n° até %d é %d\n",n, result);
    return 0;
}
```

Recursividade



NOTA: É necessário ter muito cuidado na construção de funções recursivas.

Assim:

É absolutamente necessário assegurar que existe um critério de paragem.

Este vai determinar quando a função deve parar de se invocar a ela própria.

Isto impede que a função se chame infinitas vezes, ou seja, impede que a função seja “infinita”.

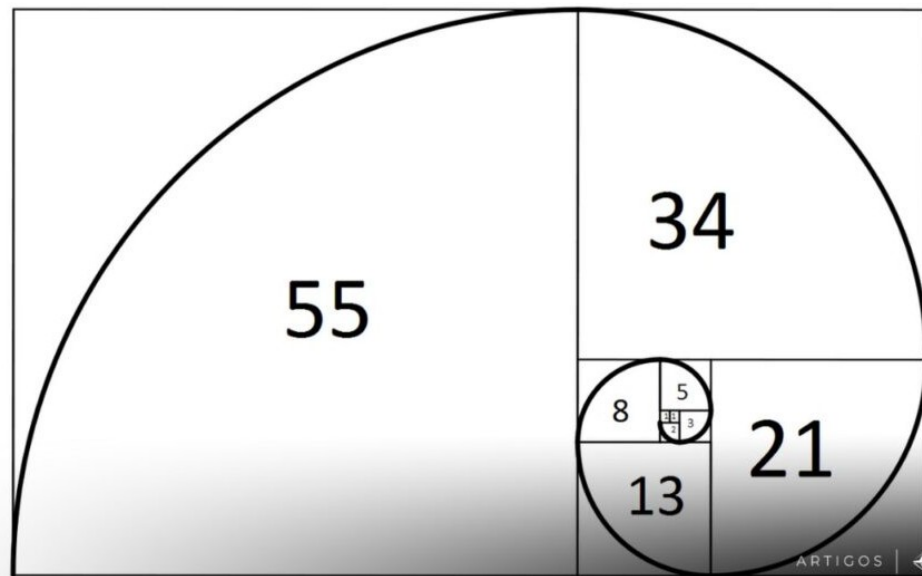


Exemplo 3: A sucessão de Fibonacci. Os números da sucessão de Fibonacci são definidos da seguinte forma:

- O primeiro número é 1.
- O segundo também é 1.
- O enésimo número é definido como sendo a soma dos dois números anteriores.

Ou seja:

se $n=1$ então $\text{fib}(n) = 1$, senão
se $n=2$ então $\text{fib}(n) = 1$, senão
 $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$





Exemplo 3: A função iterativa que implementa a sucessão de fibonacci:

```
#include <stdio.h>

int main() {
    int n, i, primeiro_termo = 0, segundo_termo = 1, proximo_termo;

    printf("Digite um número inteiro: ");
    scanf("%d", &n);

    printf("Sequência de Fibonacci de %d:\n", n);

    // Calculando a sequência de Fibonacci e exibindo na tela
    for (i = 1; i <= n; ++i) {
        printf("%d\n", primeiro_termo);
        proximo_termo = primeiro_termo + segundo_termo;
        primeiro_termo = segundo_termo;
        segundo_termo = proximo_termo;
    }

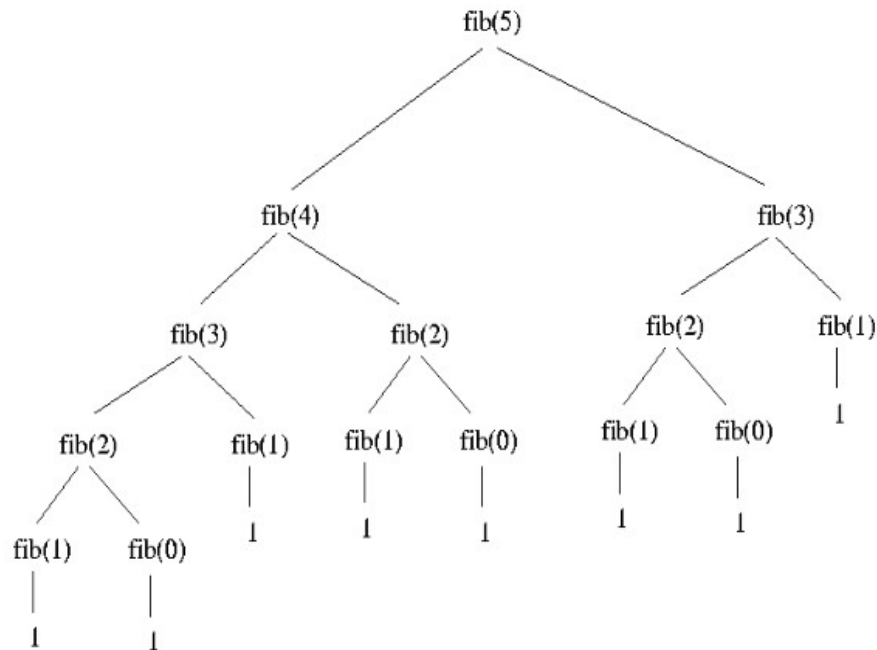
    return 0;
}
```

Recursividade



A solução recursiva é mais simples de programar do que a versão iterativa. Mas é muito ineficiente porque de cada vez que a função `fib()` é chamada, a dimensão do problema reduz-se apenas de uma unidade (de n para $n-1$), mas são feitas duas chamadas recursivas.

Isto dá origem a uma explosão combinatória e o computador acaba por efetuar os mesmos cálculos várias vezes.





Exemplo 3: A sucessão de Fibonacci

Para calcular $\text{fib}(5)$ temos de calcular $\text{fib}(4)$ e $\text{fib}(3)$.

Para calcular $\text{fib}(4)$ temos de calcular $\text{fib}(3)$ e $\text{fib}(2)$.

Para calcular $\text{fib}(3)$ temos de calcular $\text{fib}(2)$ e $\text{fib}(1)$.

.....

Este tipo de processamento é ineficiente porque obrigamos o computador a fazer trabalho “desnecessário”.

No exemplo concreto, para calcular $\text{fib}(5)$ temos de calcular: $\text{fib}(4)$ 1 vez; $\text{fib}(3)$ 2 vezes; $\text{fib}(2)$ 3 vezes; $\text{fib}(1)$ 5 vezes; $\text{fib}(0)$ 3 vezes.

Numa implementação iterativa, apenas era necessário calcular $\text{fib}(5)$, $\text{fib}(4)$, $\text{fib}(3)$, $\text{fib}(2)$, $\text{fib}(1)$ e $\text{fib}(0)$ 1 vez.



Para todo algoritmo recursivo existe um outro correspondente iterativo, que executa a mesma tarefa.

Em muitos casos a implementação iterativa é mais eficiente que a respetiva implementação recursiva.

Contudo, existem implementações recursivas que são igualmente eficientes.

Muitas vezes, é evidente a natureza recursiva do problema a ser resolvido. Por isso, a implementação recursiva pode ser muito mais simples que a correspondente implementação iterativa.

Em geral, os algoritmos recursivos possuem código mais claro (legível) e mais compacto do que os correspondentes iterativos.

Recursividade



Existem, no entanto, algumas desvantagens:

Os algoritmos recursivos quase sempre consomem mais recursos do computador, como memória, etc.

Os algoritmos recursivos tendem a apresentar um desempenho inferior aos iterativos;

Algoritmos recursivos são mais difíceis de serem depurados, especialmente quando a profundidade de recursão é elevada, ou seja, quando o número máximo de chamadas simultâneas é alto.



Quando devemos usar recursividade?

Devemos usar recursividade sempre que:

O problema é de natureza recursiva. Neste caso, a solução recursiva é mais natural e fácil de implementar dando origem a um algoritmo mais simples, elegante e fácil de compreender;

A solução recursiva não implica uma excessiva repetição do cálculo dos mesmos valores.

A solução iterativa equivalente é muito complexa, aumentando assim a probabilidade de ocorrência de erros.



Exemplo 4: cálculo da potência de um valor.

A potência de um número é o resultado da multiplicação deste número por ele mesmo várias vezes, de acordo com o expoente indicado.

Por exemplo, a potência de 2 elevado a 3 é obtida pela multiplicação de 2 por ele mesmo três vezes: $2 \times 2 \times 2 = 8$. O resultado é 8, que é o valor da potência 2 elevado a 3.

A notação matemática para indicar uma potência é:

base elevado ao **expoente**, que é escrita como "**base**^{**expoente**}".

Por exemplo, 2 elevado a 3 é escrito como " 2^3 ".



Exemplo 5: verificação se um número é primo.

Um número primo é um número inteiro maior do que 1 que possui apenas dois divisores positivos distintos, que são o 1 e ele próprio.

Por exemplo, o número 2 é um número primo, pois é divisível apenas por 1 e por ele mesmo. Já o número 4 não é primo, pois além de ser divisível por 1 e por ele mesmo, também é divisível por 2.

Portanto para verificar se um número é primo é necessário dividi-lo por todos os números anteriores. Se em nenhum desses cálculos retornar um número inteiro (resto diferente de zero), então o número é primo.



Exemplo 6: cálculo do máximo divisor comum.

o máximo divisor comum (MDC) de dois números é o maior número que divide ambos sem deixar resto. Existem duas maneiras de calcular o MDC, como por exemplo:

	1	2	2
42	30	12	6
12	6	0	
MDC (42, 30) = 6			

1) Método das divisões sucessivas: Consiste em dividir o maior número pelo menor, e depois o divisor pelo resto da primeira divisão. Repete-se o processo até que o resto seja igual a zero. O último divisor é o MDC.

Método de Euclides

2) Algoritmo de Euclides: Consiste em dividir o maior número pelo menor e tomar o resto. Em seguida, divide-se o divisor anterior pelo resto, até que o resto seja igual a zero. O último divisor diferente de zero é o MDC.

o teu • de partida



Instituto Politécnico
de Viana do Castelo

www.ipvc.pt